같은 화질에서 WebP·AVIF는 JPEG보다 얼마나 작은가

작성자: 임동호 · 작성일: 2026년 10월 14일

사진과 보안 실습 화면 캡처에서의 파일 크기 절감률 비교

초록

웹 이미지 포맷 WebP와 AVIF는 JPEG보다 파일이 작다고 알려져 있지만, 그 근거는 대부분 사진 실험이다. 이 연구는 이미지 종류와 목표 화질에 따라 절감률이 달라지는지 확인했다. Kodak 사진 24장과 보안 실습 화면 캡처 24장을 가운데 640×400으로 자른 뒤, 휘도 SSIM이 0.90, 0.95, 0.98 이상이 되는 가장 낮은 품질 값으로 JPEG·WebP·AVIF를 만들어 크기를 비교했다(총 432건, 자동화 스크립트). 사진에서는 같은 화질의 JPEG보다 WebP가 평균 25~27%, AVIF가 36~38% 작았고, 목표 화질에 따른 차이는 2%p 안팎이었다. 화면 캡처에서는 WebP가 40~55%, AVIF가 72~81% 작아 사진보다 15~45%p 컸고, 목표 화질이 높을수록 절감률이 커졌다. 가설은 일부만 뒷받침됐다. 이미지 종류의 영향은 분명했지만 목표 화질의 영향은 화면 캡처에서만 나타났고, 두 세부 예측은 방향이 반대였다. 품질 1~100 전수 탐색으로 점검한 결과 평균은 최대 1.3%p만 바뀌었고, 다른 운영체제에서 다시 실행해도 같은 파일이 나왔다.

핵심어: 이미지 압축, WebP, AVIF, JPEG, SSIM, 화면 콘텐츠

1. 서론

1.1 문제의식

이 연구의 관심 분야는 IT 개발 중 웹 이미지 최적화, 특히 이미지 압축 포맷이다. 처음에는 정보보안·네트워크 분야에서 주제를 찾았다. 이후 자동화 스크립트로 원자료를 만들 수 있는 일반 개발 주제로 범위를 옮겼다.

웹에 올리는 이미지를 작게 만드는 방법으로 JPEG 대신 WebP와 AVIF가 쓰이고, "같은 화질에서 더 작다"는 것이 주된 근거다. 그런데 이 근거가 된 실험은 대부분 사진으로 한 것이다. 보안 실습 보고서나 기술 문서에는 터미널·웹 콘솔 같은 텍스트 위주의 화면 캡처가 많이 들어간다. 이런 이미지에서도 같은 비율로 작아지는지가 이 연구의 출발점이다.

1.2 선행 연구

이미 알려진 사실은 다섯 가지다.

- Google의 연구는 Kodak 이미지 등 네 데이터셋에서, 같은 SSIM일 때 WebP가 기본 JPEG보다 약 25~34% 작다고 보고했다[2].

- Barman과 Martini는 자연·게임·합성 이미지 152장을 비교해 AVIF가 가장 효율적이며, 절감률이 이미지 종류에 따라 다르다고 보고했다[3].
- Netflix의 비교에서는 AVIF의 이점이 낮은 비트레이트에서 가장 컸다[4].
- JPEG 인코더를 mozjpeg로 바꾸면 JPEG 자체가 작아져 WebP와의 차이가 줄어든다[5], [6].
- 화면 콘텐츠는 컴퓨터가 만든 글자·그래픽과 반복 패턴이 많아 사진과 성질이 다르고[7], JPEG는 연속 색조 이미지를 대상으로 설계됐다[8].

이 연구가 새로 확인하려는 것은 세 가지다. 첫째, 터미널·콘솔 위주의 보안 실습 화면에서도 사진과 같은 절감률이 나오는가. 둘째, 목표 화질(SSIM 0.90, 0.95, 0.98)에 따라 절감률이 얼마나 바뀌며, 그 변화가 이미지 종류마다 다른가. 셋째, 이 결과를 누구나 다시 만들 수 있는 형태로 남길 수 있는가.

1.3 가설

> 이미지 종류(사진·텍스트 스크린샷)와 목표 화질(SSIM)에 따라, WebP와 AVIF가 같은 화질의 JPEG보다 파일 크기를 줄이는 비율이 달라진다.

원인은 이미지 종류·목표 화질·포맷이고, 결과는 JPEG 대비 파일 크기 절감률이다. 세부 예측은 두 가지다. 절감률은 텍스트 스크린샷보다 사진에서 크고, 목표 화질이 높을수록 작아진다. 이미지 종류별·목표 화질별 평균 절감률 차이가 5%p 미만이면 가설이 틀린 것으로 보기로, 실험 전에 정했다.

이 가설은 여러 후보 주제를 검토한 끝에 골랐다. 접은 후보는 세 가지다. "엔트로피만으로 암호화 파일을 구별할 수 있는가"는 실제 파일 묶음을 확보하기 어렵고 결론이 이미 알려져 있어 접었다. "데이터베이스 인덱스는 언제부터 손해인가"는 결과가 환경에 크게 좌우돼 접었다. "인기 사이트일수록 보안 헤더를 더 적용하는가"는 다시 수집하면 같은 결과가 나오지 않아 접었다.

1.4 가설을 다듬은 과정과 고른 이유

가설은 AI와 주고받으며 여섯 단계로 좁혔다.

1. AI에게 논문 아이디어를 요청해 보안 중심 후보 5개를 받았다.
2. 과제 조건상 원자료와 실행 절차가 필요하므로, 자동화 스크립트로 데이터를 만들 수 있는 주제로 다시 요청했다.
3. 후보 37개를 받은 뒤 "무조건 된다고 하지 말고 현실적으로" 평가해 달라고 되물었다. 이 과정에서 재현이 어렵거나 결과가 뻔한 주제를 걸렀다.
4. 보안·네트워크 외의 IT 개발 주제를 추가로 요청해 10개를 받았고, 그중 이미지 포맷 비교를 골랐다.
5. 처음 가설("WebP·AVIF가 같은 화질에서 JPEG보다 작다")은 거의 확실히 맞을 것이라 논의할 거리가 없다고 보았다. 그래서 이미지 종류와 목표 화질에 따라 절감률이 달라지는지로 좁혔다.
6. 비교 대상과 이미지 종류를 정했다. JPEG 인코더에 mozjpeg를 추가하기로 했고, 정의가 애매한 "그래픽"은 빼고 사진과 텍스트 스크린샷만 남겼다. mozjpeg는 이후 설치 문제로 제외했다(2.5절).

이 가설을 최종으로 고른 이유는 네 가지다.

- **측정할 수 있다.** 화질(SSIM)과 파일 크기를 모두 숫자로 잴 수 있다.

- **일주일 안에 데이터를 모을 수 있다.** 수집부터 측정까지 스크립트 하나로 자동화되고, 실행 시간도 몇 십 분 수준이다.
- **결과가 반대로 나와도 의미가 있다.** 절감률이 조건과 상관없이 일정하거나 JPEG가 더 작게 나오면, 통념과 다른 결과라서 그 자체로 결론이 된다.
- **재현할 수 있다.** 공개 이미지와 고정된 인코더 버전을 쓰므로, 다른 사람이 같은 결과를 다시 얻을 수 있다.

2. 방법

2.1 데이터

종류	출처	받은 날짜	장수
사진	Kodak Lossless True Color Image Suite [10], kodim01~24.png	2026-10-14	24
화면 캡처	본인이 보안 실습 중 직접 찍은 PNG (2026-09-30~10-01 촬영)	2026-10-14	후보 42 중 24

Kodak 24장은 내려받은 뒤 사이트에 적힌 바이트 수와 모두 일치하는지 확인했고, SHA-256 값을 기록했다. 화면 캡처는 CTF·GNS3·보안 관제 세 실습 묶음에서 나온 후보 42장 중, 실험 전에 정한 규칙으로 24장을 골랐다.

1. 가운데 640×400을 자를 수 없는 작은 이미지는 제외했다(3장).
2. 자른 영역이 거의 빈 화면인 것은 제외했다. 한 색이 95% 이상이거나 글자 윤곽 비율이 1% 미만인 경우다(6장).
3. 자른 영역 대부분이 사진인 것은 제외했다(1장).
4. 묶음마다 8장씩 남기고, 남는 것은 시드 2026으로 무작위 추출했다(8장 제외).

그 결과 화면 캡처 표본은 터미널·콘솔 화면 비중이 높다.

2.2 실험 설계

구분	내용
바꾼 조건	이미지 종류 2(사진, 화면 캡처) × 포맷 3(JPEG, WebP, AVIF) × 목표 SSIM 3(0.90, 0.95, 0.98)
고정한 조건	가운데 640×400 자르기, RGB 8비트, 4:2:0 색 서브샘플링, 인코더 설정(아래 표), SSIM 계산 방식

구분	내용
표본 수	이미지 48장 × 포맷 3 × 목표 3 = 432건. 조건 하나마다 이미지 24장
측정한 값	파일 크기(바이트), 선택된 품질 값, 실제 SSIM, 인코딩 시간(5회 중앙값)
주요 지표	절감률 = 1 − (WebP 또는 AVIF 크기 ÷ 같은 이미지·같은 목표의 JPEG 크기)

포맷	라이브러리 (Pillow 12.3.0 내장)	설정
JPEG	libjpeg-turbo 3.1.4	optimize=True, progressive=False, 4:2:0
WebP	libwebp 1.6.0	method=4, 손실 압축
AVIF	libavif 1.4.2 + aom 3.14.1	speed=6, 4:2:0, 스레드 1개

Kodak의 세로 이미지 6장은 원래 방향인 가로로 90° 돌린 뒤 자랐다. 사이트 설명에 따르면 원본은 모두 가로였다[10]. AVIF는 AV1으로 압축한 이미지를 HEIF 컨테이너에 담는 형식이다[9].

2.3 화질 측정과 품질 값 결정

화질은 SSIM[1]으로 재었다. 휘도(ITU-R 601-2) 채널에서 가우시안 창(σ = 1.5)을 써서 scikit-image 0.26.0으로 계산했다. 포맷마다 품질 1~100을 이분 탐색해 "SSIM이 목표 이상인 가장 낮은 품질 값"을 찾고, 그때의 파일 크기를 기록했다. 품질 1에서도 목표를 넘으면 floor로 표시했다. 이 경우 실제 절감률을 잴 수 없어 과소평가될 수 있다. 탐색 중 시도한 모든 인코딩(3,359건)은 따로 기록했다.

2.4 보조 실험

탐색 기록을 점검하다가, 품질을 올렸는데 SSIM이 내려간 인접 쌍을 91개 찾았다. 이분 탐색은 "품질이 오르면 SSIM도 오른다"는 가정에 기대므로, 이 가정이 결과에 주는 영향을 확인해야 했다. 그래서 모든 이미지·포맷을 품질 1~100으로 전부 인코딩해(14,400건), 목표 이상인 파일 중 가장 작은 것을 찾았다. 본 실험 결과는 고치지 않고 두 결과를 비교했다.

2.5 설계 변경

변경	전	후	이유
화면 캡처 출처	새로 24장 촬영	기존 실습 화면 42장 중 규칙으로 24장	실제 작업 화면을 쓰기 위해
자르는 크기	768×512	640×400	768×512 이상인 후보가 21장뿐이라 24장을 채울 수 없었음

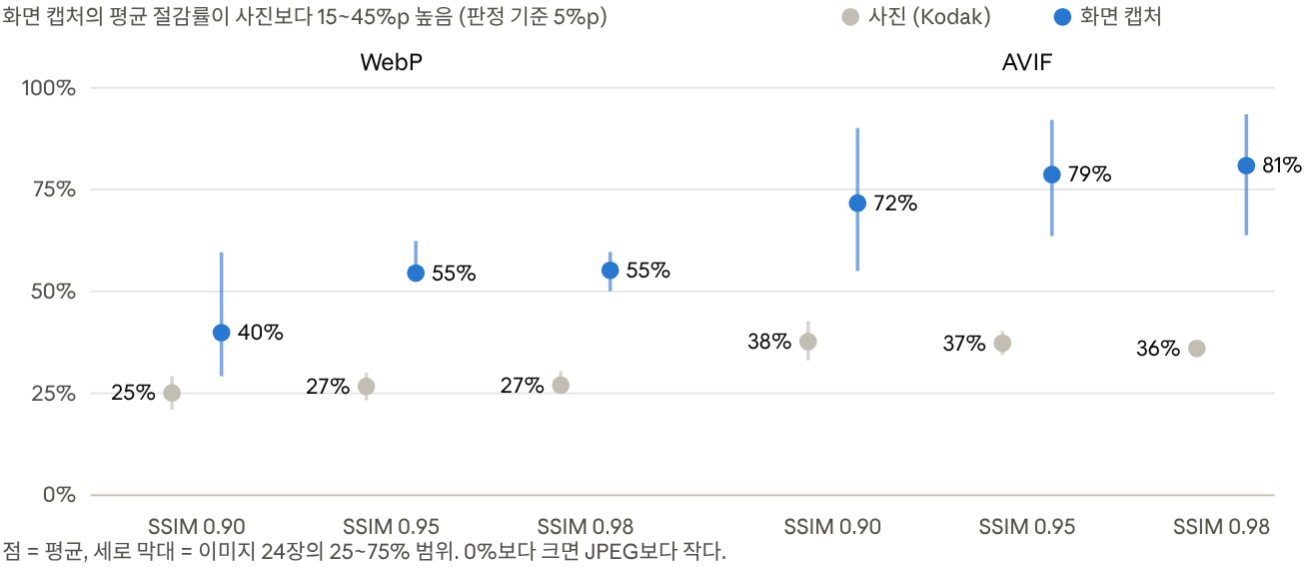
변경	전	후	이유
비교 포맷	JPEG(기본), mozjpeg, WebP, AVIF (576건)	JPEG(기본), WebP, AVIF (432건)	실험 환경에 mozjpeg를 설치할 수 없었음. "설치 안 되면 뺀다"는 규칙을 미리 정해 둠
floor 표시	없음	추가	시험 실행에서 품질 1에서도 목표를 넘는 경우를 발견
보조 실험	없음	품질 1~100 전수 탐색	본 실험 후 이분 탐색 가정이 일부 깨짐을 발견

3. 결과

3.1 절감률

432건 모두 목표 SSIM에 도달했다(unreachable 0건). 평균으로 보면 WebP와 AVIF는 모든 조건에서 JPEG보다 작았고, 그 차이는 화면 캡처에서 훨씬 컸다(그림 1).

그림 1. 같은 화질에서 JPEG 대비 절감률: 화면 캡처가 사진보다 훨씬 컸다



자료: results/savings_per_image.csv (본 실험, WebP·AVIF 288건)

사진에서는 목표 화질을 0.90에서 0.98로 올려도 평균 절감률이 WebP 1.9%p, AVIF 1.7%p만 바뀌었다. 화면 캡처에서는 같은 구간에서 WebP가 15.3%p, AVIF가 9.2%p 커졌다. 화면 캡처의 중앙값 비트율(화소당 비트)을 보면, 목표 화질을 올릴 때 JPEG는 0.64에서 1.95로 약 3배가 됐고 AVIF는 0.14에서 0.18로 약 1.3배가 됐다. 표 1은 이 결과를 판정 기준(5%p)에 비추어 정리한 것이다.

표 1. 가설대로 나온 부분과 다르게 나온 부분

가설의 부분	예측	실제	구분
이미지 종류에 따른 차이	5%p 이상	WebP 14.8~28.2%p, AVIF 33.9~44.9%p	가설대로
목표 화질에 따른 차이 (사진)	5%p 이상	WebP 1.9%p, AVIF 1.7%p	다르게 나옴
목표 화질에 따른 차이 (화면 캡처)	5%p 이상	WebP 15.3%p, AVIF 9.2%p	가설대로
세부 예측 1: 사진에서 더 큼	사진 > 화면	모든 조건에서 화면 캡처가 더 큼	다르게 나옴
세부 예측 2: 화질이 높을수록 작아짐	감소	사진은 거의 그대로, 화면은 증가	다르게 나옴

3.2 평균과 다르게 나온 개별 결과

- WebP가 JPEG보다 커진 경우가 3건 있었다. 모두 화면 캡처 SSIM 0.90이고, ctf_04(−9.9%), ctf_12(−6.8%), vpn_01(−5.8%)이다. 세 건 모두 WebP가 floor였다.
- floor는 32건으로 모두 화면 캡처에서 나왔다(WebP 15, AVIF 15, JPEG 2). floor를 빼면 화면 WebP의 평균 절감률은 50.1 / 57.5 / 55.1%가 되고, 화질에 따른 차이는 7.4%p로 줄었다. 화면 AVIF는 72.8 / 78.1 / 79.6%로 차이 6.8%p가 남았다.
- 화면 캡처의 AVIF 절감률은 자른 영역에서 가장 많은 한 색의 비율과 상관이 컸다(r = 0.68 / 0.73 / 0.77). WebP는 r = 0.07 / −0.11 / −0.47이었다.
- 세 실습 묶음 모두 사진보다 절감률이 컸다. AVIF 기준으로 CTF 54~62%, 보안 관제 74~87%, GNS3 87~94%였다.
- 인코딩 시간의 중앙값은 JPEG 0.002초, WebP 0.026초, AVIF 0.244초이었다.

3.3 보조 실험과 재현 확인

품질 1~100 전수 탐색에서는 432행 중 31행의 파일 크기가 달랐다. 모두 화면 캡처였고(AVIF 17, WebP 12, JPEG 2), 모두 더 작은 파일이 나왔다. 가장 크게 바뀐 것은 ossec_02 AVIF SSIM 0.98로 27.3% 작아졌다. 평균 절감률의 변화는 최대 1.3%p(화면 WebP SSIM 0.95, 54.5% → 55.8%)였고, 사진 72행은 모두 같았다.

재현성도 확인했다. 리눅스에서 만든 원자료 중 무작위 12건을 윈도우 PC에서 다시 인코딩했다. 같은 라이브러리 버전을 썼고, 12건 모두 바이트까지 같은 파일이 나왔다.

4. 논의

4.1 가설 판단

데이터는 가설을 일부만 뒷받침한다. 이미지 종류에 따른 절감률 차이는 WebP 14.8~28.2%p, AVIF 33.9~44.9%p로 판정 기준 5%p를 모든 조건에서 넘었다. 목표 화질에 따른 차이는 화면 캡처(WebP 15.3%p, AVIF 9.2%p)에서만 기준을 넘었고, 사진(WebP 1.9%p, AVIF 1.7%p)에서는 넘지 못했다. 두 세부 예측은 모두 방향이 반대였다(표 1).

4.2 다르게 나온 이유

검토한 가능성은 여섯 가지다.

가능성	판단	근거
A. 화면 캡처는 단색 배경이 넓어 AVIF가 적은 비트로 표현한다	받아들임	AVIF 절감률과 단색 비율의 상관 r = 0.68~0.77. 검은 콘솔 위주인 GNS3 묶음에서 87~94%
B. JPEG는 화면 캡처에서 화질을 올릴수록 비트가 급증한다	받아들임	JPEG 비트율은 약 3배, AVIF는 약 1.3배가 됐다. JPEG가 연속 색조 이미지용이라는 점[8]과 어긋나지 않는다. 다만 글자 경계의 왜곡을 직접 재지는 않았으므로 원인은 추정이다
C. floor 때문에 낮은 화질에서 절감률이 작게 잡혔다	부분적으로 받아들임	floor를 빼면 화면 WebP의 화질별 차이가 15.3%p에서 7.4%p로 줄지만 여전히 5%p를 넘는다
D. 한 실습 묶음이 결과를 끌고 갔다	버림	세 묶음 모두 같은 방향이고 크기만 달랐다
E. SSIM 0.90~0.98이 모두 높은 화질 구간이라 사진에서는 차이가 드러나지 않았다	받아들임	[2]의 Kodak 결과도 JPEG 품질 50~95에서 WebP/JPEG 크기 비가 0.70~0.73으로 거의 일정했다. [4]의 "낮은 비트레이트에서 AVIF 이점이 크다"는 이번보다 낮은 화질의 이야기일 수 있다
F. SSIM을 휘도 채널로만 계산했다	판단 보류	이번 데이터로는 확인할 수 없어 한계로 남긴다

4.3 가설 수정

- **고치기 전**: 이미지 종류(사진·텍스트 스크린샷)와 목표 화질(SSIM)에 따라, WebP와 AVIF가 같은 화질의 JPEG보다 파일 크기를 줄이는 비율이 달라진다. (세부 예측: 사진에서 더 크고, 화질이 높을수록 작아진다)

- **고친 뒤:** 이미지 종류는 WebP와 AVIF의 JPEG 대비 절감률에 크게 영향을 주어, 텍스트 위주 화면 캡처에서 사진보다 절감률이 크다. 목표 화질(SSIM 0.90~0.98)의 영향은 화면 캡처에서만 나타나며, 화질이 높을수록 절감률이 커지는 경향을 보인다.

고친 가설은 같은 데이터를 보고 만든 것이라, 이번 실험으로 검증됐다고 할 수 없다. 새 이미지로 다시 확인해야 한다.

4.4 결론

이 연구는 사진 24장과 보안 실습 화면 캡처 24장을 SSIM 0.90, 0.95, 0.98에 맞춰 압축했다. 그리고 같은 화질에서 WebP와 AVIF가 JPEG보다 파일을 얼마나 줄이는지 비교했다. 사진에서는 WebP가 평균 25~27%, AVIF가 36~38% 작았고, 목표 화질을 바꿔도 차이는 2%p 안팎이었다. 이는 Google이 발표한 "같은 SSIM에서 WebP가 25~34% 작다"는 결과[2]의 낮은 쪽과 같다.

화면 캡처에서는 절감률이 훨씬 컸다. WebP는 평균 40~55%, AVIF는 72~81% 작았으며, 사진과의 차이는 15~45%p로 판정 기준 5%p를 크게 넘었다. 또 목표 화질이 높을수록 절감률이 커졌는데, 화질을 0.90에서 0.98로 올릴 때 JPEG는 화소당 비트가 약 3배로 늘었지만 AVIF는 약 1.3배로만 늘었기 때문이다(중앙값 기준). AVIF의 절감률은 배경이 단색인 화면일수록 컸다.

따라서 가설은 일부만 뒷받침됐다. 이미지 종류의 영향은 분명했고, 목표 화질의 영향은 화면 캡처에서만 나타났다. 두 세부 예측(사진에서 더 큼, 고화질일수록 작아짐)은 모두 반대로 나왔다. 이번 표본에서는 텍스트 위주 화면을 저장할 때 AVIF가 파일 크기 면에서 가장 유리했다. 다만 인코딩은 JPEG보다 약 120배 느렸다.

4.5 한계

- 종류마다 이미지가 24장뿐이다.
- 화면 캡처는 보안 실습의 터미널·콘솔 화면 위주다. 문서나 일반 웹 화면으로 일반화하기 어렵다.
- SSIM을 휘도 채널로만 계산했고, 사람이 눈으로 본 화질 평가는 하지 않았다.
- mozjpeg를 비교하지 못했다. [6]에 따르면 mozjpeg 기준으로는 WebP의 이점이 줄어들 수 있다.
- 인코더 설정을 하나씩만 썼다(WebP method 4, AVIF speed 6).
- 이미지를 가운데 640×400으로 잘라 썼다.
- floor 32건은 실제 절감률을 정확히 재지 못했다.
- 파일 크기만 비교했고 인코딩 시간은 판정에 넣지 않았다.
- 고친 가설은 같은 데이터에서 나왔으므로 새 데이터로 검증하지 않았다.

참고문헌

모든 문헌은 2026-10-14에 확인했다. 제목으로 검색해 원문 페이지(학술지 페이지, 저자 홈페이지, 대학 저장소, 기관 공식 문서)를 열었다. 그리고 제목·저자·연도·서지 정보와, 인용한 내용이 초록이나 본문에 실제로 있는지 대조했다. 표기 형식은 APA 7판이며, 문헌마다 가설과 어떻게 이어지는지를 한 줄씩 덧붙였다.

1. Wang, Z., Bovik, A. C., Sheikh, H. R., & Simoncelli, E. P. (2004). Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing, 13*(4), 600–612. https://doi.org/10.1109/TIP.2003.819861
    - 연결: 가설의 "같은 화질"을 SSIM으로 측정하는 근거다. 주관 평가 점수가 있는 이미지 데이터베이스로 검증됐다.
2. Google. (n.d.). *WebP compression study*. Google for Developers. Retrieved October 14, 2026, from https://developers.google.com/speed/webp/docs/webp_study
    - 연결: "같은 SSIM에서 WebP가 JPEG보다 25~34% 작다"는 기존 주장이며, 사진 결과를 비교하는 기준이다.
3. Barman, N., & Martini, M. (2020). An evaluation of the next-generation image coding standard AVIF. In *2020 Twelfth International Conference on Quality of Multimedia Experience (QoMEX)*. IEEE. https://doi.org/10.1109/QoMEX48832.2020.9123131
    - 연결: 절감률이 이미지 종류에 따라 다르다는 근거다. 목표 SSIM을 정해 비교하는 방식도 이 연구와 같다.
4. Mavlankar, A., De Cock, J., Concolato, C., Swanson, K., Moorthy, A., & Aaron, A. (2020, February 13). *AVIF for next-generation image coding*. Netflix Technology Blog. https://netflixtechblog.com/avif-for-next-generation-image-coding-b1d75675fe4
    - 연결: AVIF의 이점이 낮은 비트레이트에서 크다는 관찰로, "목표 화질이 높을수록 절감률이 작아진다"는 세부 예측의 근거다.
5. Mozilla Research. (2014, March 5). *Introducing the 'mozjpeg' project*. https://research.mozilla.org/2014/03/05/introducing-the-mozjpeg-project/
    - 연결: 비교 기준인 JPEG 자체도 인코더에 따라 작아질 수 있다는 근거다.
6. Siipola, J. (2020, June 19). *Is WebP really better than JPEG?* https://siipo.la/blog/is-webp-really-better-than-JPEG
    - 연결: mozjpeg 기준으로는 WebP의 이점이 줄어든다는 반대 증거다. 개인 블로그라 한계 논의에만 썼다.
7. Liu, S., Xu, X., Lei, S., & Jou, K. (2015). Overview of HEVC extensions on screen content coding. *APSIPA Transactions on Signal and Information Processing, 4*, e10. https://doi.org/10.1017/ATSIP.2015.11
    - 연결: 화면 콘텐츠는 사진과 성질이 다르다는 근거로, 가설의 "이미지 종류" 부분과 이어진다.
8. ITU-T. (1992). *Recommendation T.81: Information technology – Digital compression and coding of continuous-tone still images – Requirements and guidelines*. International Telecommunication Union. https://www.itu.int/rec/T-REC-T.81-199209-I/en
    - 연결: JPEG가 연속 색조(사진) 이미지를 대상으로 설계됐다는 근거로, 화면 캡처에서 결과가 다를 수 있다는 예측과 이어진다.
9. Alliance for Open Media. (2025). *AV1 Image File Format (AVIF)* (v1.2.0). https://aomediacodec.github.io/av1-avif/
    - 연결: 실험 대상 포맷인 AVIF의 정의다.

10. Franzen, R. (2013). *Kodak lossless true color image suite*. https://r0k.us/graphics/kodak/
    - 연결: 사진 그룹 데이터의 출처다. [2], [4]도 같은 이미지를 써서 결과를 서로 비교할 수 있다.

목록에서 뺀 문헌

- Wallace, G. K., *The JPEG still picture compression standard*: 원문 사본에 권·호·쪽이 없어 서지 정보를 확정하지 못했다. 기대했던 "글자나 날카로운 경계에서 JPEG 성능이 떨어진다"는 내용도 원문에 없었다. 필요한 근거는 [8]로 대신했다.
- Wikipedia, "JPEG" 항목: 누구나 고칠 수 있는 백과사전이라 1차 출처가 아니어서, [8]을 찾는 데만 썼다.

부록. 재현 방법

제출 ZIP에는 이 논문, 원자료, 실행 스크립트, 화면 캡처 24장이 들어 있다. Kodak 사진 24장은 용량 제한(8MB) 때문에 넣지 않았다. 대신 내려받는 스크립트와 SHA-256 값을 넣었다.

경로	내용
README.md	재현 방법
EXPERIMENT_DESIGN.md, DESIGN_CHANGES.md	실험 설계표, 설계 변경 기록
data/	데이터 출처, Kodak 체크섬, 화면 캡처 24장, 후보 42장의 선택 기록
scripts/	Kodak 내려받기, 화면 캡처 선택, 실험, 보조 실험, 집계, 확인 스크립트
results/	원자료(raw_results.csv 432행), 탐색 기록, 보조 실험 결과, 집계 표, 실행 환경, 윈도우 PC 재현 기록

다시 확인하는 방법(Windows cmd, 파이썬 3.11 이상)은 아래와 같다.

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python scripts\fetch_kodak.py
python scripts\verify.py
```

`verify.py`는 입력 이미지가 기록과 같은지, 원자료가 432행인지 확인한다. 또 무작위 12행을 다시 인코딩해 크기와 SSIM이 같은지 비교하며, 약 1분 걸린다. 실험 전체를 다시 돌리려면 `python scripts\run_experiment.py --out results_rerun`을 실행한다(약 12분). 파일 크기는 Pillow

에 들어 있는 라이브러리 버전이 같으면 바이트까지 같게 나오며, 버전은 results/environment.json에 있다. 인코딩 시간은 컴퓨터마다 다르다.

후보 화면 캡처 42장 중 제외한 18장도 용량 제한 때문에 넣지 않았다. 따라서 24장을 고르는 과정을 처음부터 다시 돌릴 수는 없다. 다만 장마다 측정값과 선택·제외 이유는 data/screen_selection_log.csv에 남아 있다.